UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

CORSO DI GRAFICA E INTERATTIVITA'

2024/2025

Unity Scripting

Lezione 3: Hands on gaming behaviour

Docenti:

Prof. Andrea F. Abate

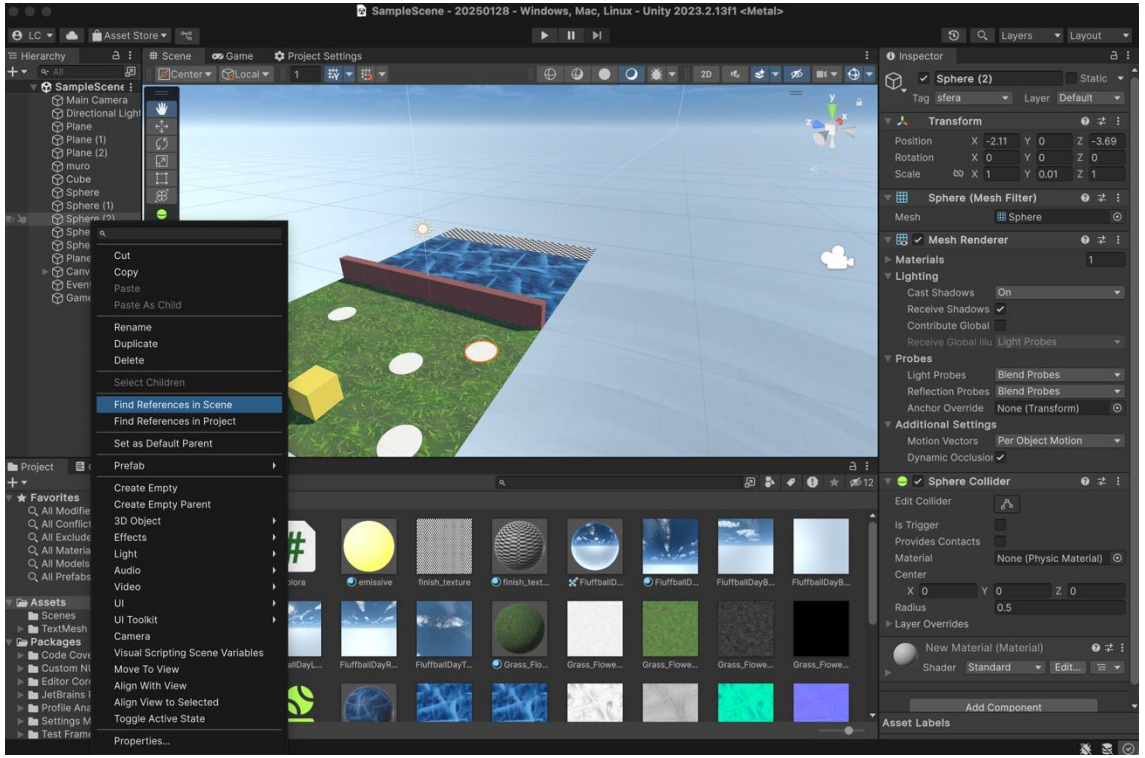
Dott.ssa Lucia Cimmino

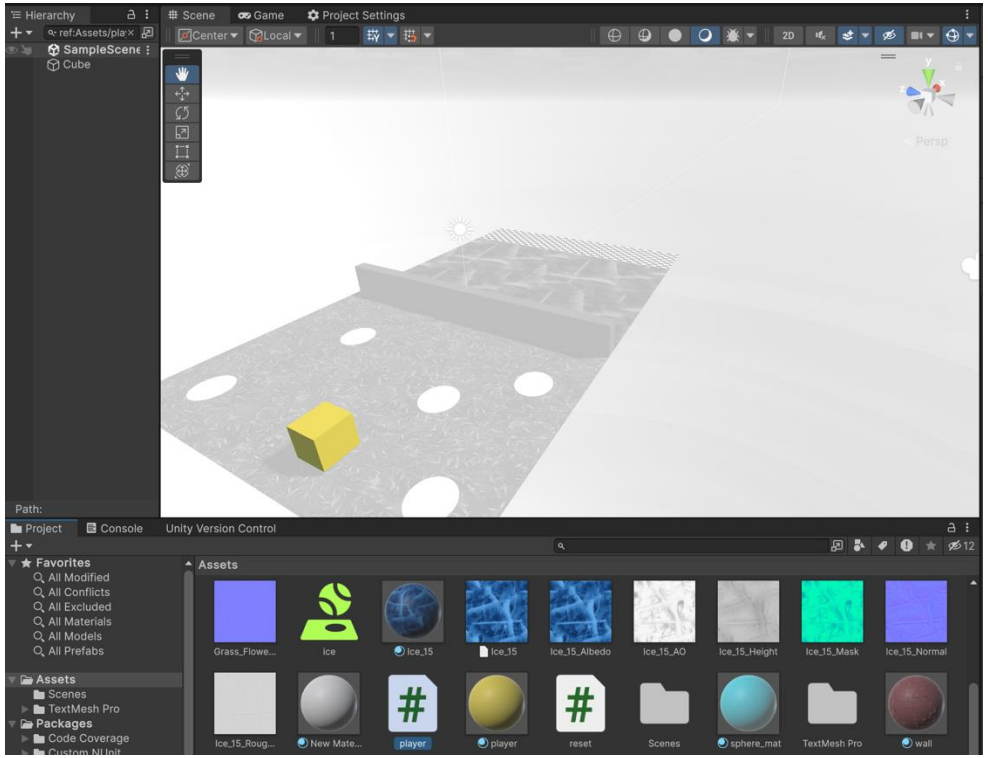
Before Starting…

Find reference in the scene

Find Reference In Scene

Useful command to **locate where a specific asset or object** is used within the scene.





Scripting

The basis

Scripting in Unity

Unity Scripting System:

- Unity uses a **custom implementation** of the **Mono runtime** for scripting (backend).
- Scripts are used to **control objects** created in the Unity Editor and the game play.

Current Language: C#

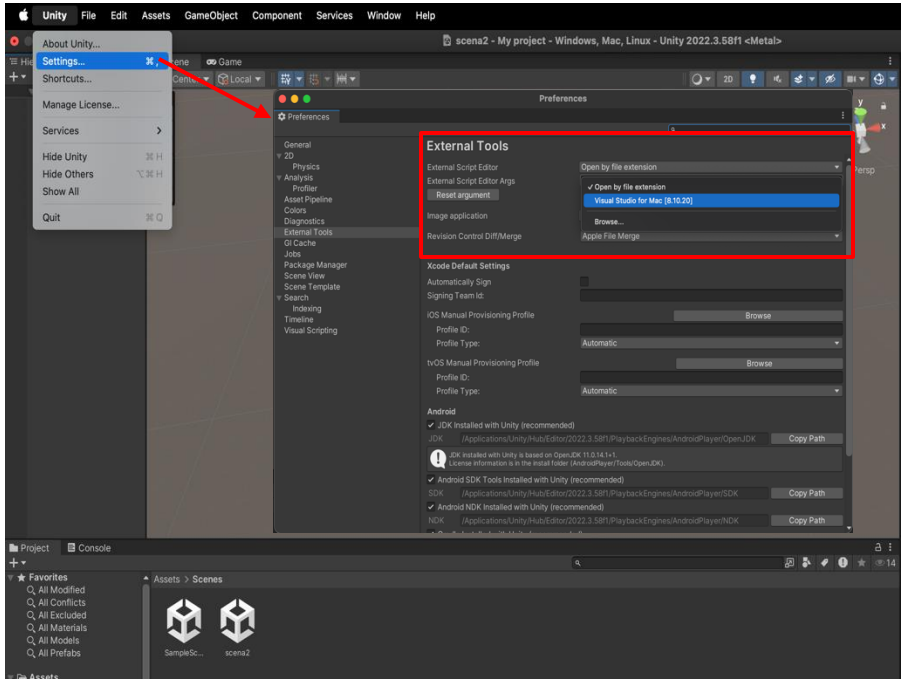
- A **modern object-oriented programming language** developed by Microsoft.
- Part of the **.NET framework**, widely used in game development.

- **Previously Supported Languages**
  - JavaScript (UnityScript): A simplified subset of JavaScript.
  - Boo: A statically-typed, Python-inspired object-oriented language.

Since 2017, **C# is the only officially supported language** in Unity.

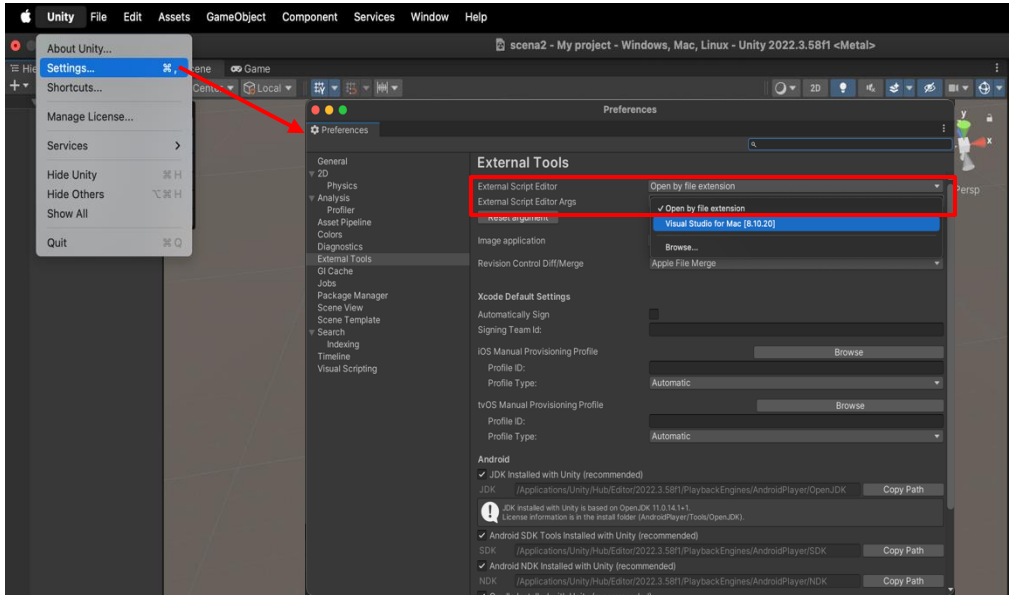
Unity Scripting: IDE

- You can set your **preferred script editor** in **Unity → Settings → External Tools**.
- Unity allows integration with popular IDEs such as **Visual Studio, Atom, Visual Studio Code** and others...

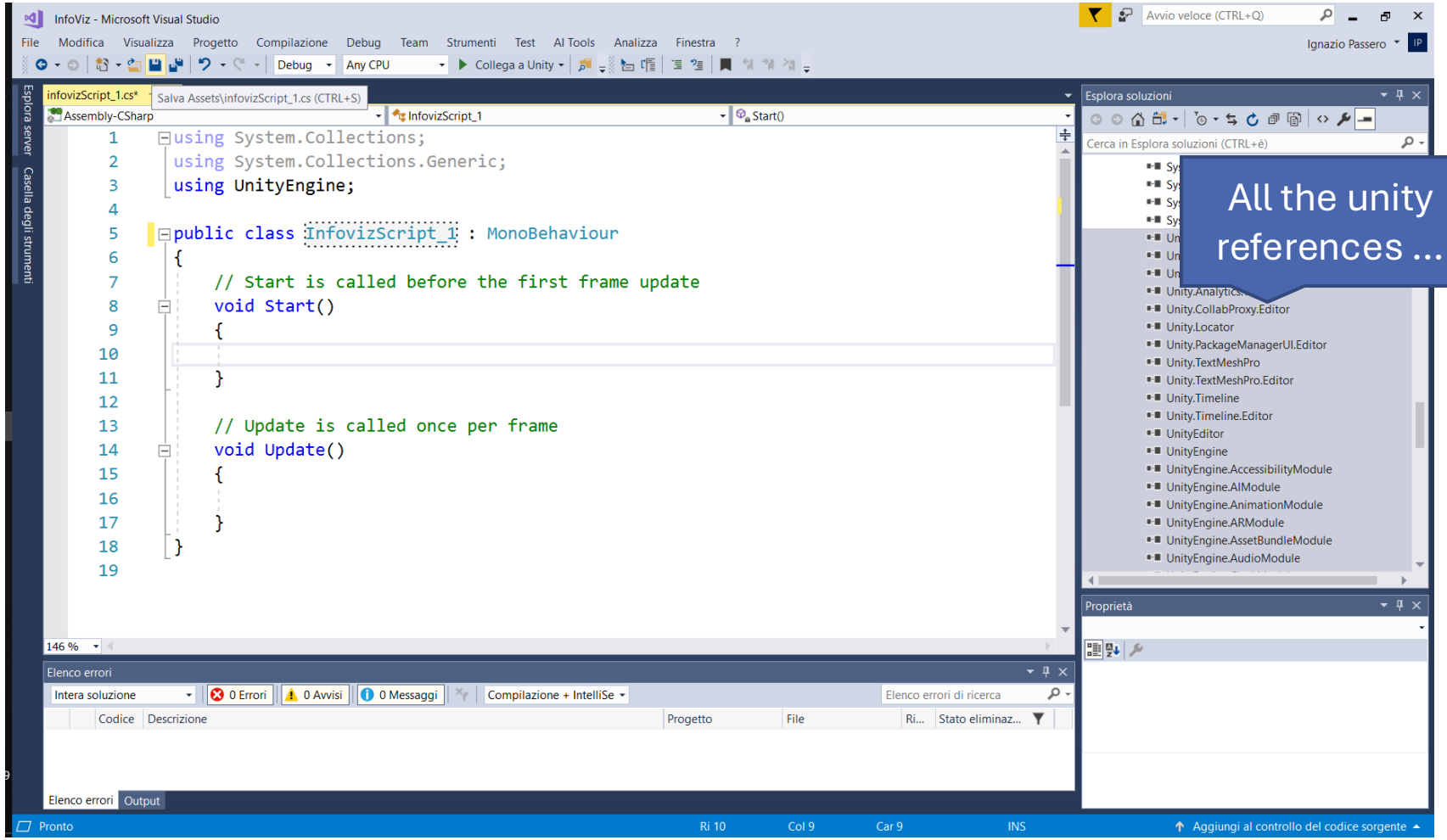


Unity Scripting: IDE

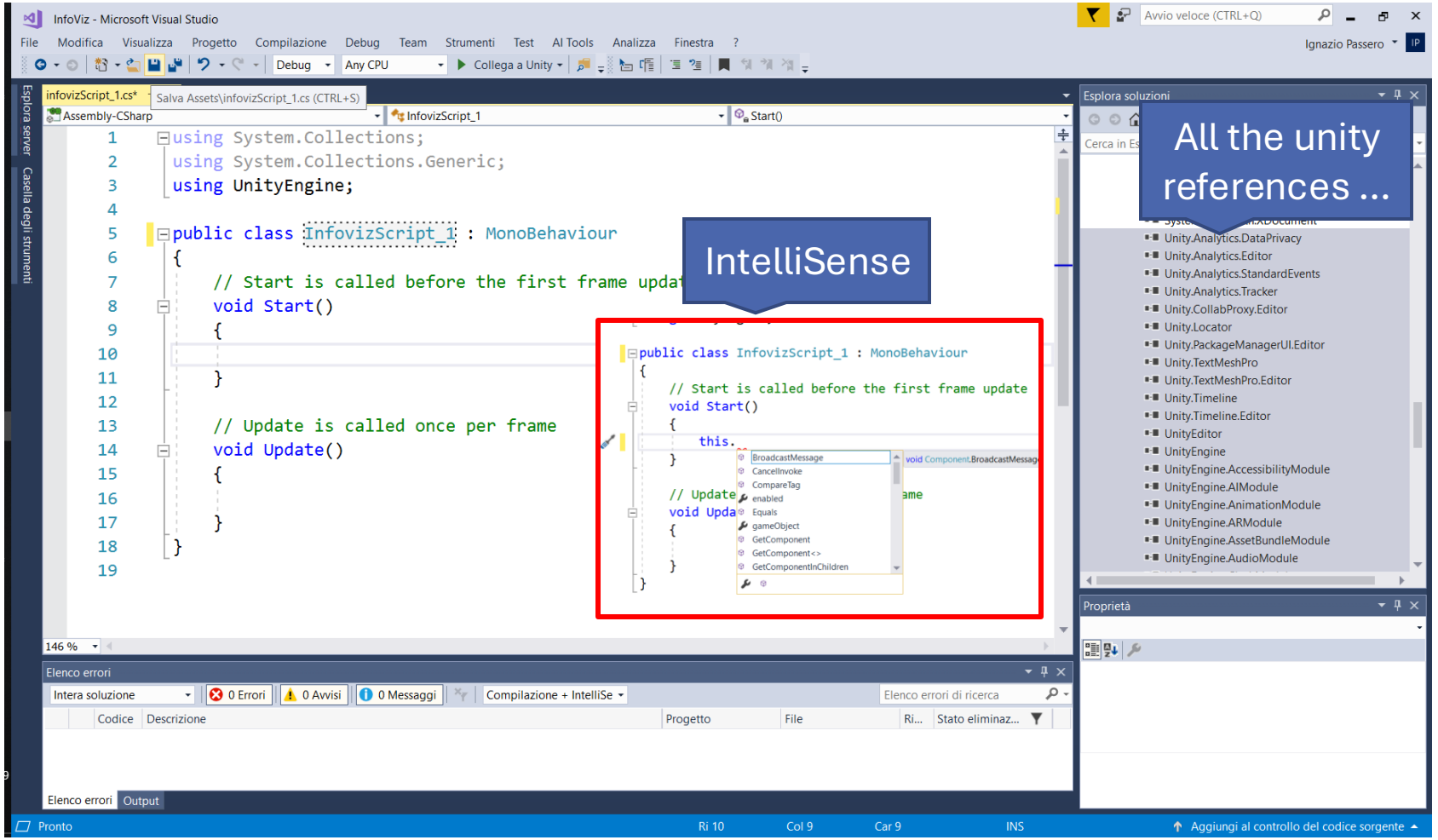
- By default, Unity uses **Visual Studio** as its script editor if it is installed.
- If no external editor is installed, Unity will default to its **built-in text editor**, which has limited functionality.



Visual Studio

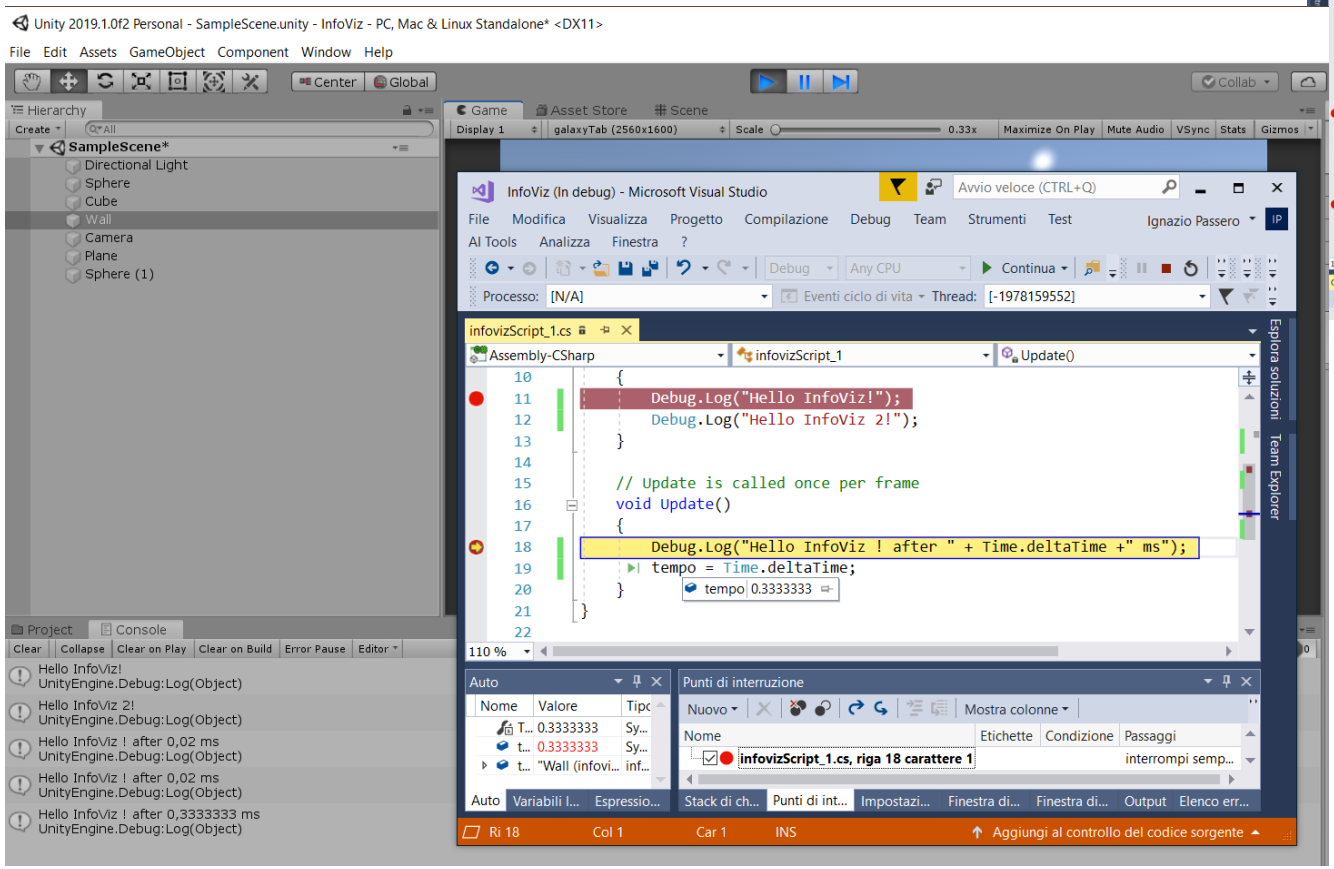


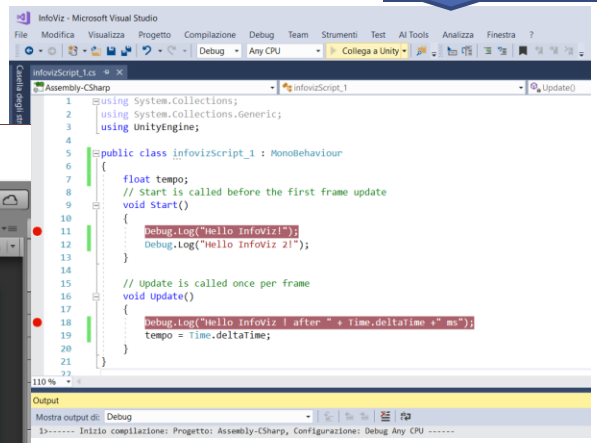
Visual Studio



Visual Studio

Unity Debug



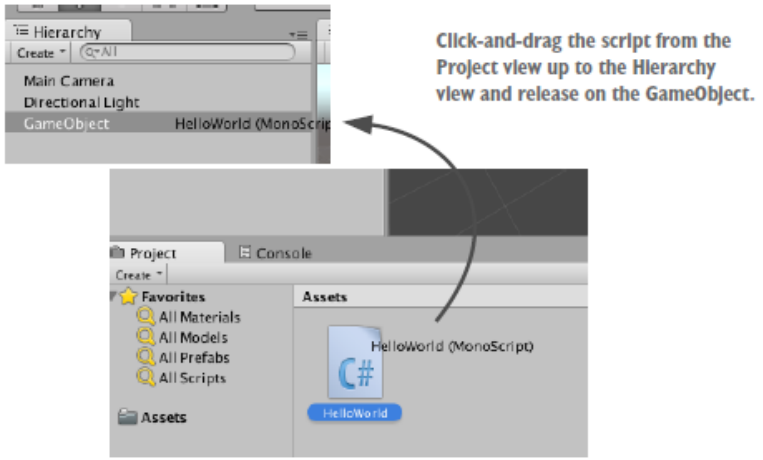


The Script Component

- In Unity, **scripts** are **components**, meaning they **must** be **attached to a GameObject** to function.

- **Creating a New Script:**
  1. In the **Project** window, go to **Assets** → **Create** → **C# Script**.
  2. Name the script appropriately (e.g., PlayerController).
  3. Double-click the script to open it in **Visual Studio** for editing.

- **Attaching a Script to a GameObject:**
  - Drag the script from the **Project view** onto the desired GameObject in the **Hierarchy**.



Scripts control **GameObjects' behavior**, just like other components (e.g., Rigidbody, Collider).

A typical C# “empty” Script

```
using UnityEngine;
using System.Collections;

public class ScriptName : MonoBehaviour
{
  // Use this for initialization
  void Start () {
        initializations
  }

  // Update is called once per frame
   void Update () {
        do something in the current frame
  }
}
```

Names consistency

```
using UnityEngine;
using System.Collections;

public class Move : MonoBehaviour
{
  // Use this for initialization
  void Start () {
        initializations
  }

  // Update is called once per frame
   void Update () {
        do something in the current frame
  }
}
```

!! Keep the class and the script name Consistent !!

When you rename a script, you need to manually update the class name in the script to match

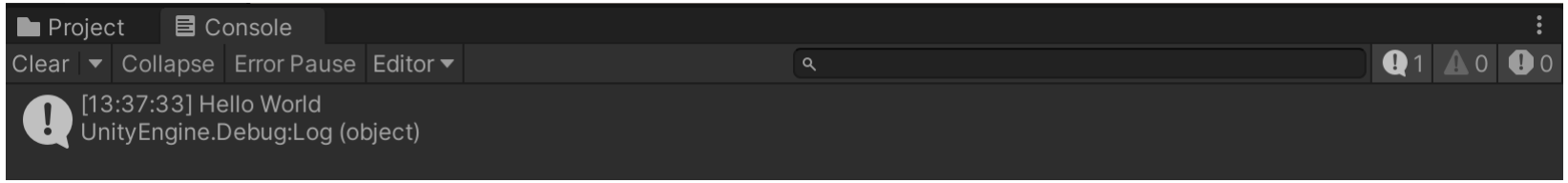
Let’s start coding

Listing 1.2 Adding a console message

```
void Start() {
    Debug.Log("Hello World!");
}
```

Add the logging command here.

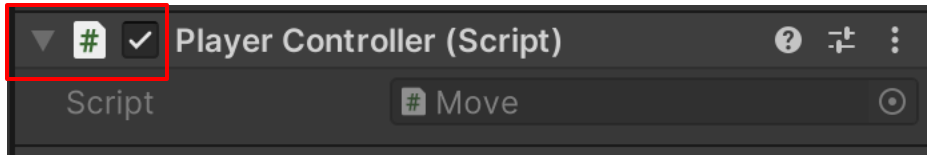
- The **Start() method** is called **once** when you press **Play** in the Unity editor.
- After adding a **log command** to your script, **save** the script before running the scene.
- Press **Play**, then switch to the **Console** window to see the message **"Hello World!"** appear.
- The **Console** is useful for debugging and tracking script execution in Unity.



A more complete C# Script

- **MonoBehaviour** is the **base class** from which all Unity scripts derive.
- When coding in **C#**, you must **explicitly inherit** from MonoBehaviour to access built-in lifecycle methods.
- Unity provides a **checkbox** in the **Inspector** to **enable or disable** a MonoBehaviour script. When **unchecked**, the script's functions are **not executed**, effectively disabling the component.
- However, if none of the following lifecycle methods are present in the script, **the checkbox does not appear** in the Inspector:

Start( )
Update( )
FixedUpdate( )
LateUpdate( )
OnGUI( )
OnDisable( )
OnEnable( )



A more complete C# Script

```
using UnityEngine;
using System.Collections;

public class MainPlayer : MonoBehaviour
{
  // Called one on script loading
  void Awake() {
        initializations
  }
  // Use this for initialization
  void Start () {
        initializations
  }
// Update is called once per frame
   void Update () {
        do something in the current frame
  }
  // FixedUpdate is called after each Physics update
   void FixedUpdate () {
        do something after Physics update
  }
  // LateUpdate is called after Update and
  FixedUpdate
   void LateUpdate () {
        do something after Update/FixedUpdate
  }
}
```

https://docs.unity3d.com/ScriptReference/MonoBehaviour.html

Awake() and Start()

- Both methods are used for **initialization**, but they have key differences in when they are called.
- ***Awake()***
  - Called **when the script is loaded**, before the scene starts running.
  - Executes **even if** the **script component is disabled** in the Inspector.
- ***Start()***
  - Called **just before the first frame update.**
  - Executes **only if** the **script component is enabled** in the Inspector.

Key Difference: Awake() runs as soon as the script loads, even if disabled, while Start() only runs if the script is enabled.

Update(), FixedUpdate() and LateUpdate()

- These methods are used for **continuous execution** during gameplay, but each serves a different purpose.
- ***Update()*** is called every frame, if MonoBehaviour is enabled.
- ***FixedUpdate*** is called after each Physics update.
- ***LateUpdate*** is called after Update function every frame, if MonoBehaviour is enabled.

Update(), FixedUpdate() and LateUpdate()

- ***Update()***
  - Called **once per frame**, as long as the **MonoBehaviour is enabled**.
  - Used for **game logic, user input, and animations**.
  - Frame rate dependent → execution time **varies based on FPS**.

Update(), **FixedUpdate()** and LateUpdate()

- **FixedUpdate()**
  - Called **at a fixed time interval**, **independent of frame rate**.
  - Used for **physics calculations** (e.g., Rigidbody movement).
  - May be called **multiple times per frame** if the frame rate is low, ensuring consistent physics behaviour.

Update(), FixedUpdate() and **LateUpdate()**

- **LateUpdate()**
  - Called **after Update()** each frame, if the **MonoBehaviour is enabled**.
  - Used for **camera movements** or actions that should occur **after all Update logic is processed**.
  - Ensures smooth following behaviour when tracking objects.

Update(), FixedUpdate() and LateUpdate(): in Summary

Method	When is it called?	Best Used For
Update()	Every frame (frame rate dependent)	Game logic, input handling, animations
FixedUpdate()	At a **fixed** time step (physics engine update)	Physics calculations, Rigidbody movement
LateUpdate()	After Update() (every frame)	Camera following, final adjustments

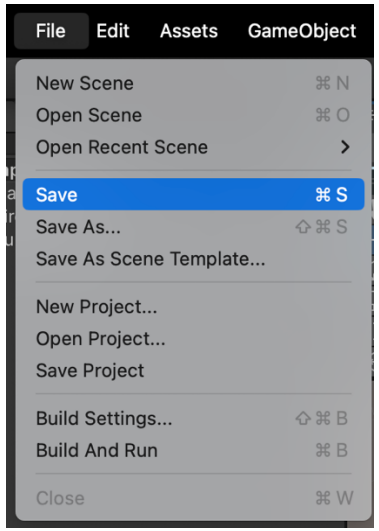
Saving and Exporting a Scene

How to save

Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓SAVE THE SCENE!

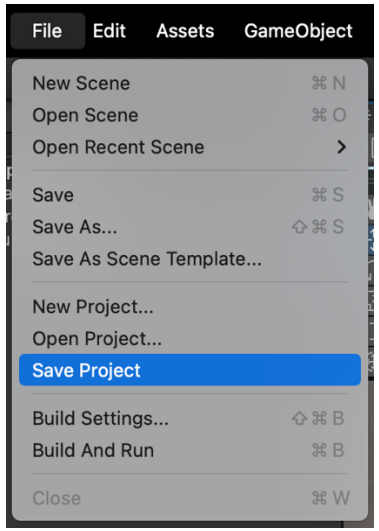
- Go to **File → Save** or press **Ctrl + S (Windows) / Cmd + S (Mac)**.
- This saves the current scene, including objects, lighting, and layout.



!! If you don't save the scene, any changes will be lost when Unity closes **!!**

✓SAVE THE PROJECT!

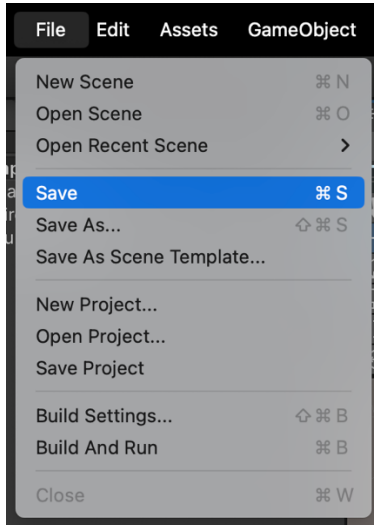
- Go to File → Save Project.
- This saves **project-wide settings**, such as imported assets, prefab modifications, and package settings.



How to save

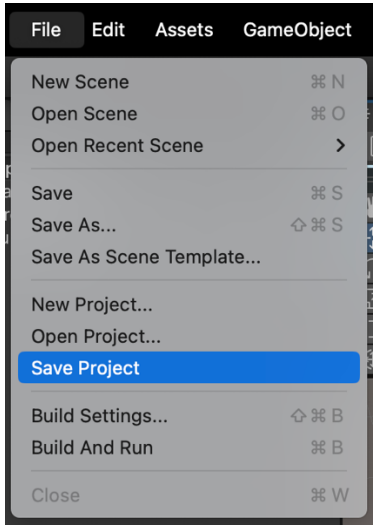
Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

SAVE THE SCENE!



!! If you don't save the scene, any changes will be lost when Unity closes !!

SAVE THE PROJECT!

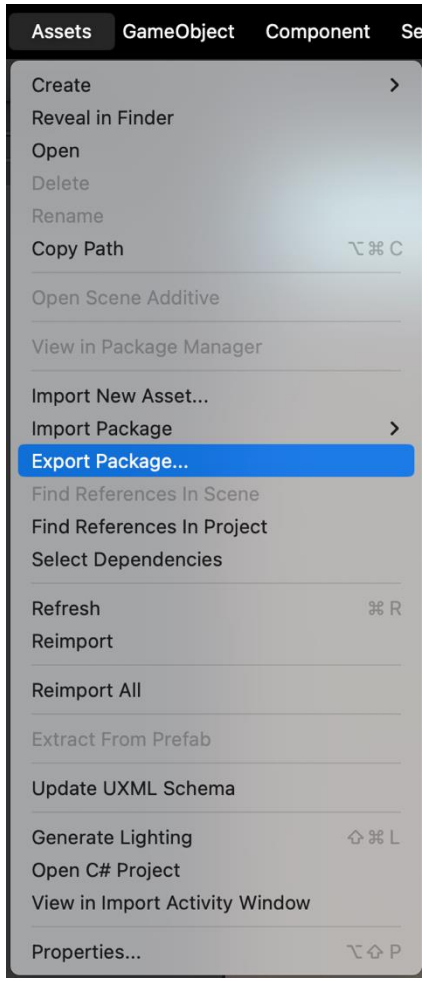


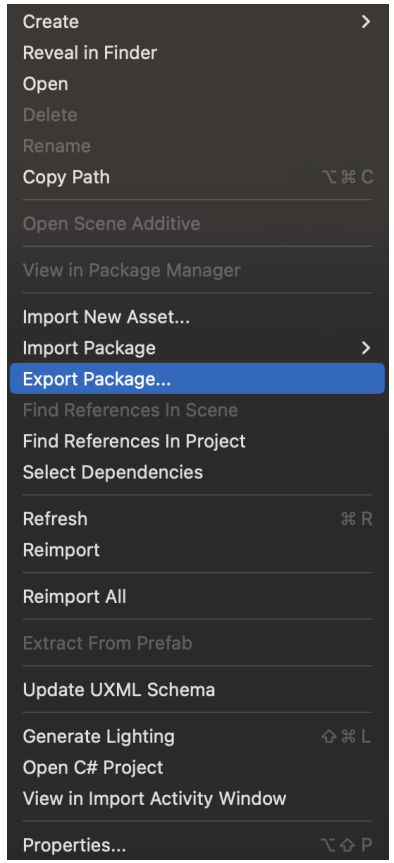
NOTE! Using "Save Project" **does not save** changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

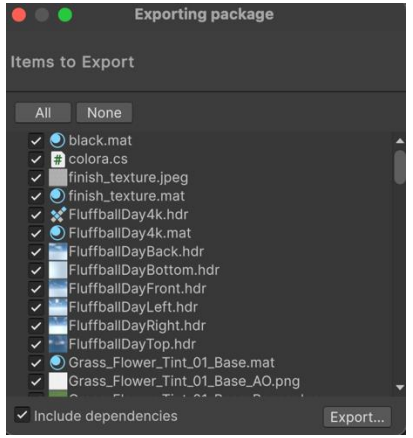
Project Export

If you want to transfer your assets into a different project and preserve all the project's information, you can export your **assets** as a **Custom Package**.

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package…** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.







Project Export

WORNING: When exporting the project, make sure you are in the **correct folder**. To export the entire Assets folder, you must be inside the Assets folder. If you select a subfolder, only the elements within that subfolder will be exported, and the rest will not be exported!!!

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package…** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.